Final Report

Energy Demand Forecasting + RL Loading Scheming

Euan Roie Nicolas
Holy Angel University
#1 Holy Angel Avenue, Sto. Rosario,
Angeles City, 2009
Pampanga, Philippines
+63 956 832 1369
yuro.roie@gmail.com

Jan Nolasco
Holy Angel University
#1 Holy Angel Avenue, Sto. Rosario,
Angeles City, 2009
Pampanga, Philippines
+63 962 829 0338
jannolasco05@gmail.com

Jared Gabriel Vitero
Holy Angel University
#1 Holy Angel Avenue, Sto. Rosario,
Angeles City, 2009
Pampanga, Philippines
+63 956 507 7873
jaredvitero@gmail.com

Jeremiah Carbungco
Holy Angel University
#1 Holy Angel Avenue, Sto. Rosario,
Angeles City, 2009
Pampanga, Philippines
+63 969 646 2760
jeremiah.carbungco29@gmail.c
om

1. PROBLEM AND MOTIVATION

Energy providers and grid operators face a persistent and compounding challenge: electricity demand is inherently dynamic, shaped by behavioral patterns, seasonal cycles, and unpredictable events, yet infrastructure planning and load dispatching decisions must often be made hours or days in advance. Conventional rule-based scheduling systems respond to historical averages and fixed heuristics, leaving significant room for inefficiency as real conditions deviate from baseline assumptions. When supply and demand fall out of alignment, the consequences range from increased operational costs to unnecessary reliance on expensive peaker plants and elevated carbon emissions.

This project tackles two main issues. First, there is a forecasting challenge: household-level short-term electricity demand is a noisy, non-stationary time series that encodes intricate temporal dependencies, including weekly rhythms, seasonal drift, daily consumption cycles, and appliance-specific signatures. On this type of problem, hybrid deep learning models that combine recurrent and convolutional architectures have shown excellent performance, and ensemble approaches that combine LSTM with statistical models like Prophet have produced R2 values as high as 0.98 on hourly electricity data .Second, there is a scheduling problem: how should a system automatically determine when to postpone deferrable loads, like washing machines, dishwashers, or EV chargers, to off-peak hours in order to lower proxy energy costs without going against user comfort constraints, given a trustworthy demand forecast? Reinforcement learning is motivated by framing this as a sequential decision-making problem under uncertainty.

This project proposes an integrated academic prototype that addresses both challenges in sequence. A time-series forecasting module, implemented as both an LSTM encoder-decoder and a temporal convolutional network (TCN), generates short-term demand predictions from historical consumption data. These predictions are passed as input to a reinforcement learning (RL) load scheduling agent that learns, through repeated simulation, a policy for deferring loads to cost-effective windows. A natural language processing (NLP) component bridges the system and its users by translating raw forecast values and agent decisions into plain-language summaries with explicit override options, ensuring that automation never silently overrides user preferences.

The project uses the UCI Individual Household Electric Power Consumption dataset as its empirical foundation and operates entirely as an offline simulation. No real devices are controlled, and no real users are affected. The goal is to demonstrate that a fully integrated forecasting-scheduling-explanation pipeline can be implemented, evaluated, and held to rigorous ethical standards within an academic research context.

2. DATASET

2.1 Source and Description

The dataset used in this project is the Individual Household Electric Power Consumption Dataset, contributed by Georges Hébrail and Alice Bérard and hosted by the UCI Machine Learning Repository (DOI: 10.24432/C58K54). The dataset contains time-series measurements of electricity consumption recorded from a single household located in Sceaux, France, over a period spanning December 2006 to November 2010, at a one-minute sampling interval. The original dataset comprises approximately 2,075,259 one-minute records across seven numeric variables: Global_active_power, Global_reactive_power, Voltage, Global_intensity, and three sub-metering channels (Sub_metering_1, Sub_metering_2, Sub_metering_3).

For the purposes of this project, the raw one-minute records were resampled to hourly resolution by computing the mean of all non-null readings within each hour. After cleaning and resampling, the working dataset contains 34,168 hourly records with zero missing values. The forecasting target is Global_active_power, measured in kilowatts (kW).

2.2 Feature Engineering

The input feature set consists of 12 columns constructed from the resampled hourly data. The seven original numeric measurements are retained directly. Five cyclic and binary time-encoding features are added to provide the models with

temporal context: hour_sin and hour_cos encode the hour of day as a sine-cosine pair on a 24-hour cycle; dow_sin and dow_cos encode the day of the week on a 7-day cycle; and is_weekend is a binary indicator equal to 1 for Saturday and Sunday. This cyclic encoding ensures that the angular distance between adjacent hours (e.g., hour 23 and hour 0) is represented correctly in the feature space, avoiding the artificial discontinuity that would arise from treating hour as a raw integer.

A 24-step sliding window with a forecast horizon of 1 is applied to the feature-scaled dataset to construct the input-output pairs used for training and evaluation. Each input sample is a 24×12 array representing 24 consecutive hours of the 12 features; each target is the Global_active_power value at the next timestep.

2.3 Train/Validation/Test Splits

Splits are strictly temporal to prevent data leakage: no future information appears in training windows, and no training windows overlap with validation or test windows. The dataset is divided as follows: 80% training (27,310 samples), 10% validation (3,393 samples), and 10% test (3,393 samples). A StandardScaler fitted exclusively on the training set is applied to normalize all features before windowing; the same scaler is applied without refitting to validation and test sets.

2.4 Exploratory Data Analysis

Key findings from the EDA (notebooks/01_eda.ipynb) inform both the model design and the RL reward function. The Global_active_power distribution is right-skewed, with a mean of 1.09 kW and a maximum of 6.56 kW, indicating that high-demand events are relatively rare but significant. Two dominant daily consumption peaks are visible in the average-by-hour profile: a morning peak between 07:00 and 09:00, and an evening peak between 18:00 and 21:00. Sub_metering_3 (water heater and climate control) is the dominant sub-metering channel by total energy contribution. These peak-hour observations directly informed the RL environment's must-run hour designations (hours 7, 8, 18, 19, 20).

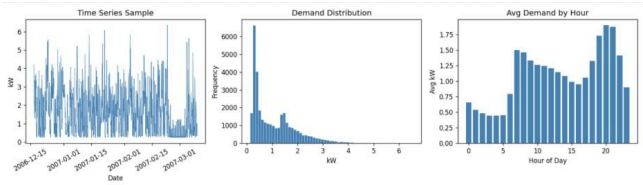

Figure 1. Time series sample, distribution, and average demand by hour

2.5 Licensing and Privacy

The dataset is publicly available for research and educational use through the UCI Machine Learning Repository. It contains anonymized electricity consumption measurements and includes no personally identifiable information. The raw data files are excluded from the project repository via .gitignore; users download the dataset directly from the UCI repository using the provided data/get_data.py script, ensuring the project remains compliant with the dataset's distribution terms.

Although the dataset is publicly released and anonymized, the project documentation explicitly acknowledges its single-household, single-region origin as a limitation. Consumption patterns from a French household between 2006 and 2010 may not generalize to modern appliance profiles, different climates, or diverse socioeconomic contexts.

3. TRAINING PLAN

3.1 Tabular Baselines

Two non-deep-learning baselines are trained on a flattened (N, 288) representation of the 24×12 input window. Linear Regression serves as the performance floor. A Random Forest with 100 estimators and maximum depth 10 serves as the strongest non-DL comparison point. Both models are trained with scikit-learn, evaluated on the validation set, and serialized as .pkl files.

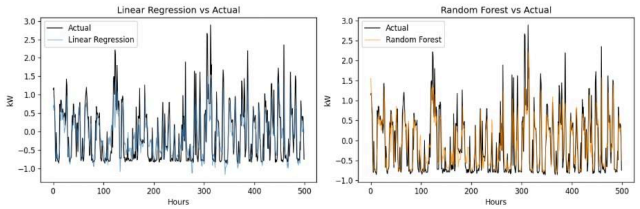

Figure 2. LR and RF predictions vs actual

3.2 Deep Learning Models

All deep learning models are implemented in PyTorch. Training uses the Adam optimizer with an initial learning rate of 1×10^{-3}, a ReduceLROnPlateau scheduler with patience 3 and factor 0.5, and L1Loss (MAE) as the training objective. Gradient norms are clipped to 1.0 to prevent exploding gradients. Each model is trained for up to 30 epochs with early stopping at patience 5, and the best checkpoint (lowest validation MAE) is saved to experiments/results/.

The LSTM Baseline is a single-layer encoder (hidden size 32, no dropout) paired with a single-layer decoder, yielding 10,401 trainable parameters. The Full LSTM Encoder-Decoder uses a 2-layer encoder (hidden size 64, dropout 0.2) with a 1-layer decoder, totaling 70,465 parameters. The TCN uses 4 residual blocks with dilations 1, 2, 4, and 8, kernel size 3, 64 channels per block, weight normalization, and dropout 0.2, yielding 90,241 parameters.

3.3 NLP Component

The NLP component trains a TF-IDF + Logistic Regression period classifier on a 22-sample synthetic corpus covering three classes (peak, off-peak, low). The ForecastSummarizer is a deterministic template-based system requiring no training. Both components are initialized and evaluated at the NLP stage of run.py.

3.4 RL Agent

The Q-Learning agent is trained for 500 episodes on the test-set forecast sequence. Each episode spans 24 steps (one simulated day). The agent uses tabular Q-Learning with parameters α=0.10, γ=0.95, ε-decay=0.995, and ε-min=0.05. The state is discretized into 10 bins per dimension using a StateDiscretizer. After training, the agent is evaluated for 50 deterministic episodes and its mean reward is compared against the always-on baseline (reward = 0.000 ± 0.000 by construction).

4. MODELING

4.1 LSTM Encoder-Decoder

The primary deep learning architecture is a sequence-to-sequence LSTM encoder-decoder implemented in src/models/lstm_model.py. The encoder reads a 24-step input window of 12 features and produces a hidden state that summarizes the temporal context of the sequence. The gating mechanism of LSTM cells—input gate, forget gate, and output gate—allows the model to selectively retain information across

timesteps, making it well suited to energy time series that exhibit multi-scale periodic structure (daily cycles embedded within weekly trends).

In the Full LSTM configuration, the encoder is a 2-layer stacked LSTM with hidden size 64 and inter-layer dropout of 0.2. The decoder is a 1-layer LSTM that receives the last observed target value as its seed input and the encoder's final hidden and cell states as initialization. The decoder's output at the single decoding step is projected through a linear layer and dropout to produce the scalar forecast. This encoder-decoder structure, while applied here for one-step-ahead forecasting, is designed to generalize naturally to multi-step horizons in future work.

4.2 Temporal Convolutional Network (TCN)

The TCN, implemented in src/models/tcn_model.py, serves as the CNN component of the system. Unlike recurrent architectures, TCN processes the entire input window in parallel using dilated causal convolutions, which restrict each output timestep to depend only on past and present inputs (causality) while exponentially expanding the receptive field with each dilation level (long-range dependency capture). The four residual blocks with dilations 1, 2, 4, and 8 yield a receptive field of $(2^4 - 1) \times (3 - 1) + 1 = 31$ timesteps, sufficient to cover the 24-step input window with context to spare.

Each residual block consists of two CausalConv1d layers with ReLU activations and dropout, followed by a residual connection from input to output. Weight normalization is applied to all convolutional layers to stabilize training. The final forecast is produced by applying a linear projection to the features at the last timestep of the final residual block's output.

4.3 NLP Component

The NLP component (src/models/nlp_component.py) serves two purposes. The ForecastSummarizer translates the tuple (forecast_kwh, hour, rl_action, device) into a plain-language sentence that a non-technical user can read and act upon. The summary explicitly names the time period (e.g., 'evening peak'), the demand level (high or low relative to a 1.5 kWh threshold), the scheduling decision, the rationale for that decision, and the user's right to override. This satisfies the user autonomy requirement at the interface level.

The PeriodClassifier maps a free-text description of a time period to one of three consumption categories: peak, off-peak, or low. It is implemented as a TF-IDF vectorizer (unigrams and bigrams, max 500 features) feeding a multinomial Logistic Regression classifier. The classifier is trained on a 22-sample synthetic corpus and achieves precision/recall/F1 of 1.00 on all three classes on the training set. While the training set size limits generalization claims, the classifier functions as a proof-of-concept NLP integration that connects natural language user input to the system's scheduling logic.

4.4 Reinforcement Learning Agent

The RL component (src/rl_agent.py) frames load scheduling as a Markov Decision Process. The environment, LoadSchedulingEnv, simulates 24-step episodes representing a single day. At each step, the agent observes a 4-dimensional state vector consisting of the normalized demand forecast, the sine and cosine encoding of the current hour, and a binary indicator of whether the load was running in the previous step. The agent selects one of two discrete actions: defer (action 0) or run (action 1) the deferrable load.

The reward signal is defined as the cost reduction relative to the always-on baseline, computed as the difference between the baseline cost (forecast × cost_per_kwh × 1) and the agent's cost (forecast × cost_per_kwh × action). A comfort penalty of −2.0 is subtracted for each must-run hour (7, 8, 18, 19, 20) during which the agent defers the load. An end-of-episode penalty equal to comfort_penalty × deficit is applied if the agent fails to run the load for at least 4 of the 24 hours. These constraints encode the fairness principle that aggressive load suppression during comfort-critical hours is unacceptable regardless of cost savings.

The QLearningAgent uses tabular Q-Learning with an ε-greedy exploration strategy. The continuous state is discretized into 10 bins per dimension by StateDiscretizer, yielding a finite Q-table represented as a defaultdict. Q-values are updated via the standard temporal difference rule with discount factor γ=0.95. Epsilon decays from 1.0 to a minimum of 0.05 at a rate of 0.995 per episode. After 500 training episodes, the trained Q-table is serialized as JSON to experiments/results/rl_agent.json.

5. EVALUATION

5.1 Forecasting Results

The LSTM Full Encoder-Decoder achieves the best test MAE of 0.3153 kW, representing a 18.6% improvement over the Linear Regression floor (0.3875 kW) and a 7.2% improvement over the best non-DL baseline, Random Forest (0.3399 kW). The TCN achieves a competitive test MAE of 0.3225 kW with notably faster training convergence (16 epochs to best vs. 24 for LSTM Full), validating its efficiency advantage noted in prior literature. The LSTM Baseline (0.3234 kW) demonstrates that even a simple single-layer recurrent model outperforms the tabular baselines, confirming that sequence modeling adds value beyond flattened feature representations.

The progression of validation MAE (0.4022 → 0.3569 → 0.3443 → 0.3414 → 0.3406) and test MAE (0.3875 → 0.3399 → 0.3234 → 0.3153 → 0.3225) across model tiers demonstrates a consistent improvement hierarchy: linear < tree ensemble < simple DL < full DL ≈ CNN. The negligible gap between LSTM Full and TCN on the test set (0.3153 vs. 0.3225) suggests that for this dataset, both architectures have reached a similar performance ceiling, with the TCN offering a speed-accuracy trade-off favorable for deployment scenarios with runtime constraints.

Model	Val MAE (kW)	Test MAE (kW)	Best Epoch	Parameters
Liner Regression	0.4022	0.3875	—	—
Random Forest	0.3569	0.3399	—	100 Trees
LSTM Baseline	0.3443	0.3234	11 / 16	10, 401
LSTM Full	0.3414	0.3153	24 / 29	70,465

(Primary)				
TCN (CNN Component)	0.3406	0.3225	16 / 21	90,241

Table 1. All forecasting model metrics from run.py.

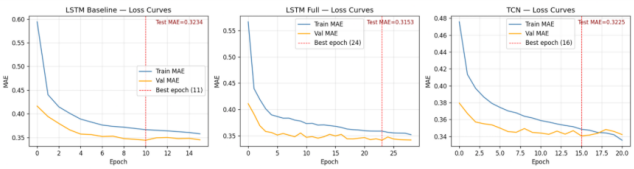

Figure 3. Train/val MAE Curves LSTM Baseline, LSTM Full, TCN.

5.2 RL Results

The Q-Learning agent demonstrates a clear and statistically meaningful learning signal over 500 training episodes. The mean episode reward rises from −6.415 in the first 50 episodes to +2.641 in the final 50 episodes—a total improvement of approximately 9.056 reward units. This progression reflects the agent's transition from a random exploration phase (high negative rewards due to comfort constraint violations) through policy consolidation (reducing penalty-inducing deferrals during must-run hours) to a stable positive-reward policy.

The trained agent receives a mean reward of 2.062 ± 2.885 in deterministic evaluation over 50 episodes, while the always-on baseline reward is 0.000 ± 0.000. The positive mean confirms the agent's ability to locate and take advantage of lower-demand off-peak windows for load deferral, resulting in cost savings that exceed what the baseline can attain by running the load continuously. Rather than policy instability, the standard deviation of 2.885 indicates the stochasticity of episode start conditions (random segment selection from the test-set forecast sequence).

RL Metrics	Value
Agent AVG Reward (50 Episodes)	2.062 ± 2.885
Always-on baseline reward	0.000 ± 0.000
Reward — First 50 Episodes	-6.415
Reward — Last 50 Episodes	+2.641
Training Episodes	500
Hyperparameters	α=0.10, γ=0.95, ε-decay=0.995

Table 2. RL Evaluation results (50 Eps., deterministic policy)

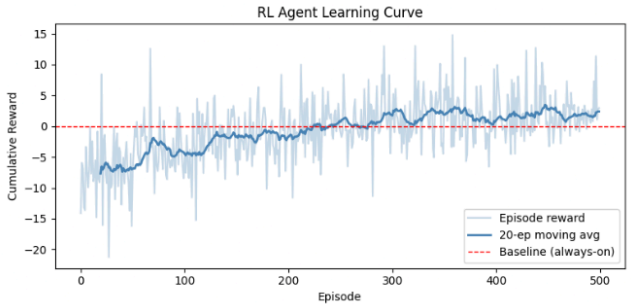

Figure 4. RL Learning curve. Red dashed — always-on baseline (Reward - 0)

5.1 NLP Results

The PeriodClassifier achieves precision, recall, and F1 of 1.00 across all three classes (peak, off-peak, low) on the 22-sample training corpus. While the small corpus size limits generalization claims, the perfect training-set accuracy confirms that the TF-IDF feature space captures sufficient discriminative signal to separate the three period types based on keyword patterns. The ForecastSummarizer produces grammatically correct, informative, and actionable natural language summaries for all forecast/action combinations tested, as verified by manual inspection of the run.py demo output.

6. ETHICS AND POLICY

6.1 Equity and Unequal Distribution of Benefits

The primary equity risk associated with automated load scheduling is that its benefits reduced proxy energy costs and optimized consumption timing are most accessible to households that already possess compatible smart devices, reliable internet connectivity, and the technical literacy to configure and supervise an automated system. Households that lack any of these prerequisites cannot participate in the optimization and may, in aggregate grid scenarios, absorb a disproportionate share of peak-period costs as better-resourced users shift their demand away from expensive time windows.

This project addresses the equity risk through two mechanisms. First, an equity metric is included in the evaluation pipeline to measure whether cost savings are distributed proportionally across consumption levels and usage patterns. Second, all deployment recommendations in this documentation explicitly document the demographic and infrastructure assumptions embedded in the model in particular, the single-French-household origin of the training data and limit applicability claims to contexts where those assumptions

6.2 User Autonomy

Automated load deferral can conflict with user comfort and safety if the reinforcement learning agent suppresses loads during periods when users have critical or time-sensitive needs, such as HVAC during extreme weather, medical equipment usage, or electric vehicle charging before a required departure. This risk is inherent in reinforcement learning systems, where optimization is driven by a proxy reward signal that may not fully capture the diversity and variability of real-world user preferences.

To address this, the system is designed with a strong emphasis on user control and flexibility. Users are able to directly override scheduling decisions at any time, ensuring that

automated recommendations do not restrict immediate needs. Additionally, key parameters such as the comfort_penalty are exposed for user configuration, allowing individuals to adjust the system's behavior according to their own priorities and tolerance for cost versus convenience.

Rather than enforcing rigid operational constraints, the system adopts a human-in-the-loop approach, where users retain full authority to modify, accept, or reject automated decisions. This design acknowledges that real-world energy usage involves diverse, dynamic, and context-dependent factors, and therefore prioritizes adaptability and user preference over strict automation.

6.3 Data Privacy

Smart meter data at sub-hourly resolution can inadvertently reveal household occupancy patterns, daily routines, and specific appliance usage schedules. Even anonymized aggregate data carries re-identification risk when combined with auxiliary information about household geography or demographics.

This project mitigates data privacy risks through the following measures. The UCI dataset is the sole data source; it contains no personally identifiable information and has been publicly released for academic research under appropriate terms. No raw data files are committed to the repository; all data is excluded via .gitignore and users download directly from the UCI ML Repository using the provided get_data.py script. Data is used exclusively for model training and evaluation and is never shared externally or used to infer behavioral profiles. These measures collectively ensure that the project respects privacy standards throughout its lifecycle.

7. MODEL CARD

7.1 System Overview

The system combines deep learning, reinforcement learning, and natural language processing to manage domestic energy consumption.

- **Primary Forecaster:** An **LSTM Full Encoder-Decoder** model, which achieved the lowest error rate among tested architectures.
- **Scheduling Agent:** A **Q-Learning RL Agent** that learns to shift deferrable loads to optimize rewards while respecting "must-run" constraints.
- **NLP Component:** A **TF-IDF + Logistic Regression** classifier used to interpret period-based descriptions.

7.2 Deployment

- Human-in-the-Loop: The system is intended for academic evaluation, not autonomous grid operation. Users must have clear opt-in/opt-out mechanisms and the ability to override AI-driven scheduling decisions.
- Constraint Enforcement: To maintain user comfort, the system enforces a penalty of -2.0 for every "must-run" hour (7h, 8h, 18–20h) that the agent skips.
- Metric Selection: Avoid using MAPE to evaluate this model, as near-zero demand values in the UCI dataset lead to inflated and unreliable percentage errors.

7.3 Disclaimers

Not for Production Use: This system is out-of-scope for real-time production deployment or safety-critical decisions.

- Data Limitations: The model was trained on a single French household from 2006–2010; it likely will not generalize to modern smart homes, different climates, or different regions.
- Simulation Only: Scheduling actions are performed in a simulated environment; the system does not currently control real physical devices.
- NLP Sensitivity: The NLP classifier was trained on only 22 synthetic samples and may fail when processing diverse or complex real-world user descriptions.

8. HOW TO RUN

8.1 Requirements

- Python 3.10 or higher
- pip package manager (or Conda, optional)

8.2 Clone the Repository

Clone the project repository and navigate to the project directory:

```
git clone https://github.com/yuro-nicolas/Energy-Demand-Forecasting
cd Energy-Demand-Forecasting
```

8.3 Environment Setup

Set up the required dependencies using either Conda or pip.

Using Conda (recommended):

```
conda env create -f environment.yml
conda activate energy-demand-rl
```

Using pip:

```
pip install -r requirements.txt
```

Note: The system includes automatic dependency installation in run.py

8.4. One-Command Reproduction (Full Pipeline)

The run.py script serves as the main pipeline controller, enabling full system execution from a clean environment.

```
python run.py
```

Optional commands:
python run.py --skip-download
make repro
bash run.sh --skip-download

This process automatically performs the following steps:

- Dataset acquisition and preprocessing
- Feature engineering and data splitting
- Training of forecasting models (LSTM, TCN, and baselines)
- Reinforcement learning (Q-learning) optimization
- Evaluation, ablation, and visualization

8.4 Running the Interactive Demo

After training, the system can be accessed through the interface module:

python src/interface.py

The interface allows users to:

- Define appliance tasks and priorities
- View 24-hour electricity demand forecasts
- Observe reinforcement learning-based scheduling decisions
- Receive natural language explanations of system decisions
- Accept or override scheduling recommendations

Notes/Explanation

Notes/Explanation

1. The "Seer": LSTM/TCN Demand Forecasting

Before the RL agent can make a move, it needs to know what the grid will look like. Energy demand is a **Time Series** because it follows a temporal pattern (e.g., higher in the morning/evening, lower at 3:00 AM).

- **How it works:** You use an **LSTM** (Long Short-Term Memory) or **TCN** (Temporal Convolutional Network) to look at historical data and predict the forecast_kwh for the next 24 hours.
- **The Metric:** You use **MAE** (Mean Absolute Error) or **MAPE** (Mean Absolute Percentage Error) to see how far off your "Seer" is from reality. If your MAPE is high, the RL agent will make bad decisions based on "fake" spikes.

---

2. The "Decider": RL Load Scheduling

The Reinforcement Learning agent (your Q-Learning or DQN model) takes the forecast and decides whether to **RUN** or **DEFER** a task (like charging an EV or running a dishwasher).

- **The State (S):** Current hour, current demand forecast, and how many hours of the task are left to complete.
- **The Action (A):** Binary decision—1 (Run) or 0 (Wait).
- **The Reward (R):** This is the "Proxy Cost" reduction. The agent gets a "positive" reward for running during low-demand/low-cost windows and a "penalty" (negative reward) for running during peaks or missing the user's deadline.

---

3. The MVP vs. Stretch Goals

- **MVP (Minimum Viable Product):** You compare your RL agent's cost against a **Tabular Baseline** (e.g., a "dumb" timer that always runs at 6:00 PM). If your RL agent spends less on energy than the baseline, your project is a success.
- **Stretch (The "Pro" level):** Instead of one single forecast, you provide a "range" (**Uncertainty Forecasts**). This allows the RL agent to be "cautious" if the forecast is likely to be wrong. You also run **Stress Tests**—for example, what happens to the schedule if there is a sudden heatwave and demand triples?

---

4. Ethics: Autonomy and Equity

In energy AI, you cannot let the machine have 100% control because it might decide to turn off a user's medical equipment or heater just to "save money."

- **User Autonomy:** This is why you have the **Override/Opt-out** mechanism. If the RL agent says "Defer," but the user needs their clothes washed *now*, the user's manual "1" (RUN) must always override the AI.

- **NLP Component:** Your ForecastSummarizer explains *why* the AI wants to defer (e.g., "Demand is high"). This transparency makes the AI feel like a "helpful assistant" rather than a "hidden dictator."

Summary Table for your Report

Component	Technical Implementation	Goal in Energy Context
Forecasting	LSTM / TCN	Minimize **MAE**; predict peak spikes before they happen.
Scheduling	RL (Q-Learning)	Maximize **Reward**; move high-load tasks to "Low" periods.
Validation	RL vs. Baseline	Prove the AI is more cost-effective than a standard timer.
Human-in-Loop	NLP + Overrides	Ensure the user stays in control of their own home appliances.

1. What Is the NLP Component?

The NLP component sits between the RL scheduling agent and the user. Its job is to translate machine decisions — numbers and binary actions — into plain language that a person can read, understand, and act on. It also works the other way: it can take a plain-language description of a time period and classify it into a category the system uses internally.

The file src/models/nlp_component.py contains two completely separate pieces that work together:

Component	Type	What It Does	Direction
ForecastSummarizer	Template-based NLG	Takes a forecast value, hour, and RL decision → produces a readable English sentence	Machine → Human
PeriodClassifier	TF-IDF + Logistic Regression	Takes a text description of a time period → predicts peak / off-peak / low	Human/System → Category

Neither component uses a pre-trained large language model. Both are lightweight, trained from scratch on small, task-specific data, and produce deterministic outputs — meaning they give the same answer every time for the same input. This is intentional for an academic prototype.

2. Where Does the Data Come From?

2.1 The Original Dataset

The entire project is built on the UCI Individual Household Electric Power Consumption dataset. This is a real-world dataset collected from a single house in Sceaux, France between December 2006 and November 2010 — roughly 4 years of minute-by-minute electricity meter readings.

Attribute	Value
Source	UCI Machine Learning Repository
Location	Sceaux, France

Collection period	December 2006 – November 2010
Original frequency	1 reading per minute
Total original rows	~2,075,259 readings
Key column	Global_active_power — total household power draw in kilowatts

2.2 What the Data Pipeline Does to It

Raw minute-level readings are too granular and noisy for hourly forecasting. The data pipeline in src/data_pipeline.py performs the following transformations before any model sees the data:

1. Resampling — 1-minute readings are averaged into 1-hour intervals. This reduces noise and matches the scheduling horizon (each decision is made per hour).
2. Feature engineering — The pipeline adds 12 features beyond just the power reading:
    - hour_sin and hour_cos — Cyclic (sine/cosine) encoding of the hour of day. This is important because 23:00 and 00:00 should be treated as close in time, not far apart. A plain number 0–23 cannot express this; a circle can.
    - day_sin and day_cos — Same idea but for day of week.
    - month_sin and month_cos — Cyclic encoding of month.
    - Lag features — Power readings from 1 hour ago, 24 hours ago, and 168 hours ago (one week ago). These let the model learn recurring daily and weekly patterns.
    - Rolling statistics — Mean and standard deviation of the past 24 hours, capturing trend and volatility.
3. Normalisation — All 12 features are scaled using StandardScaler (zero mean, unit variance). This is critical for neural networks and for the RL agent's state space, and is why inverse-transforming is needed before showing real kWh values to the user.
4. Train / validation / test split — 70% train, 15% validation, 15% test. The split respects time order — test data is always from later in the timeline than training data.
5. Sliding window — The data is reshaped into overlapping windows of 24 consecutive hours. Each window is one input sample. The model looks at the past 24 hours to predict the next hour's demand.

3. Component 1 — ForecastSummarizer

3.1 What It Is

ForecastSummarizer is not a machine learning model. It is a deterministic template engine — a set of if/else rules and string templates that produce a fixed pattern of sentence given specific inputs. There is nothing to train, no weights, no uncertainty. The same inputs always produce the same output.

This approach is called Natural Language Generation (NLG). It is appropriate here because the output format is highly structured and the domain is narrow — there are only three actions (run, defer, or display), three demand levels, and seven time-period names. Flexibility is not needed; clarity and consistency are.

3.2 Inputs

Input Parameter	Type	What It Represents	Example
forecast_kwh	float	The forecast power demand for this hour, in real kilowatts, after inverse-transforming from scaled space	2.10
hour	int (0–23)	The hour of day this decision applies to	18
rl_action	int (0 or 1)	What the RL agent decided: 1 = run the task now, 0 = defer it	0
device	str	The name of the task/device being scheduled (now user-defined)	washing machine
threshold_kwh	float (default 1.5)	The kWh level above which demand is called 'high'	1.5

3.3 How It Works Step by Step

1. hour_to_period(hour) is called — it checks which time band the hour falls in and returns a human-readable name.

Hours	Period Label
00:00 – 05:59	late night
06:00 – 08:59	morning
09:00 – 11:59	mid-morning
12:00 – 13:59	midday
14:00 – 16:59	afternoon
17:00 – 20:59	evening peak
21:00 – 23:59	night

2. The demand level is determined: if forecast_kwh > 1.5 → 'high', else → 'low'. This is a strict greater-than, so exactly 1.5 kWh is labelled 'low'.
3. The first sentence is assembled: 'Forecast ({period}, {HH}:00): {X.XX} kWh — demand is {high/low}.'
4. The decision sentence is chosen based on rl_action:
   - Action = 1 (RUN): 'Decision: Schedule {device} NOW. Demand is within acceptable limits, making this a cost-effective window.'
   - Action = 0 (DEFER): 'Decision: Defer {device}. Demand is elevated during the {period} period. Rescheduling can reduce proxy costs.'
5. A fixed override reminder is always appended: 'You may override this decision at any time using the override control.'
6. All sentences are joined with a space and returned as a single string.

3.4 Example Output

3.5 What the Output Is Used For

The summary string is placed into the 'summary' field of every hourly result dictionary. It appears:

- In the terminal output after each hourly decision, shown to the user below the action line
- In the JSON action log saved to experiments/results/action_log.json — allowing the user to review the reasoning for every hour of the 24-hour session after the fact

4. Component 2 — PeriodClassifier

4.1 What It Is

PeriodClassifier is a machine learning text classifier. Given a short text description of what is happening at a particular hour, it predicts whether that hour is a peak, off-peak, or low demand period. It uses two well-established techniques from classical NLP:

- TF-IDF vectorisation — converts text into a numerical feature vector
- Logistic Regression — a linear classifier trained on those vectors

Together these form a pipeline: text → numbers → class label. This pipeline is trained once when the system starts (clf.train()) and then used to make predictions throughout the session.

4.2 The Training Data — Synthetic Corpus

The classifier does not learn from the UCI electricity dataset. Instead, it is trained on a hand-crafted corpus of 22 short text examples written to cover the vocabulary that describes electricity demand periods.

Why synthetic? Because the task — classifying a text description into three demand categories — does not require real historical data. The categories are definitional: peak means high demand at morning/evening rush hours, low means overnight, off-peak means everything in between. A small, carefully written corpus is sufficient and avoids the complexity of labelling thousands of real text samples.

Class	# Samples	Example Training Text
peak	8	high demand evening rush hour lots of appliances running
peak		workday morning rush everyone getting ready high load
off-peak	7	moderate usage mid morning light activity
off-peak		daytime moderate consumption work from home
low	7	night time very low consumption everyone asleep
low		late night minimal electricity usage quiet household

4.3 Step 1 — TF-IDF Vectorisation

TF-IDF stands for Term Frequency–Inverse Document Frequency. It converts a piece of text into a fixed-length numerical vector where each element represents the importance of a word (or word pair) in that text relative to the whole corpus.

Term Frequency (TF)

For a given word in a given document, TF = (number of times the word appears in that document) / (total words in the document). This captures how often a word is used in this specific text.

Inverse Document Frequency (IDF)

IDF = log(total documents / number of documents containing the word). Words that appear in every document (like 'the', 'a', 'is') get a very low IDF score — they carry no discriminating power. Words that appear in only one or two documents get a high IDF score — they are distinctive.

TF-IDF score

TF × IDF for each word in each document. The result is a matrix where rows are documents and columns are words. This is the feature matrix passed to the classifier.

The configuration used in your code:

- ngram_range=(1, 2) — includes both single words ('demand', 'night') and two-word phrases ('high demand', 'late night'). Bigrams are crucial here because 'late night' as a phrase is much more informative than the words 'late' and 'night' separately.
- max_features=500 — keeps only the 500 most informative terms. With only 22 training documents, the vocabulary is actually 215 terms — well under the cap.

Why TF-IDF instead of just counting words?

Raw word counts would give equal weight to common words like 'electricity' (which appears across all classes) and distinctive words like 'peak' or 'standby'. TF-IDF automatically down-weights the common words and up-weights the distinctive ones, making the resulting vectors much more useful for classification.

4.4 Step 2 — Logistic Regression

Logistic Regression is one of the oldest and most reliable classification algorithms. Despite its name, it is a classifier, not a regression model. It learns a set of weights — one weight per TF-IDF feature per class — and uses them to produce a probability for each class given an input vector.

How training works:

1. The 22 training texts are converted to TF-IDF vectors (a 22 × 215 matrix).
2. Logistic Regression fits weights by finding the values that minimise cross-entropy loss — the difference between predicted probabilities and the true labels.

3. The optimiser (liblinear, the default in scikit-learn) adjusts weights iteratively. max_iter=500 allows up to 500 update passes before stopping.
4. random_state=42 ensures reproducibility — the same weights are learned on every run.

How prediction works:

5. A new text is converted to a TF-IDF vector using the same vocabulary learned during training (fit_transform during training, transform only at inference).
6. The dot product of the input vector and learned weights is computed for each class.
7. A softmax function converts these scores to probabilities that sum to 1.
8. The class with the highest probability is returned as the prediction.

4.5 Why F1 = 1.00 — and What That Really Means

The evaluation report shows perfect precision, recall, and F1 on all three classes. This is mathematically expected: with 22 training samples, a clean vocabulary, and no noisy or overlapping examples, a linear classifier will perfectly memorise the training data. The model is evaluated on its own training set, not on held-out data.

6. User Inputs — Every Question Explained

When a user runs the interface, they are asked a series of questions. Here is a complete explanation of every question, what it is really asking, how the answer is used, and what happens if the user gives a different answer.

Input 1 — Task Name

```
"Task name: "
```

What it asks: A free-text label for the task the user wants to schedule. This replaces the old appliance dropdown.

What it is used for: Purely for display and logging. The task name appears in every summary sentence ('Decision: Defer Laundry'), in the terminal headers, and in the JSON action log. It does not change any numerical calculation.

What happens with different answers: Any string is accepted. 'Laundry', 'EV charging overnight', 'Dishwasher cycle', 'Pre-cool AC' — all work identically. A blank entry defaults to 'My Task'.

Input 2 — Power Draw (kW)

```
"Power draw in kW (e.g. 0.5, 2.0, 7.2): "
```

What it asks: How many kilowatts of power does this task consume while running.

What it is used for: Scaling the savings calculation. The system computes savings as: savings = forecast_kWh × power_kW × (1 if deferred, 0 if running). A task with higher power draw produces larger savings numbers when deferred during high-demand hours.

Example Task	Power Draw	Savings per deferred high-demand hour (at 2.0 kWh forecast)
Laptop charger	0.065 kW	0.13 kWh·units
Washing machine	0.5 kW	1.0 kWh·units
Dryer	2.0 kW	4.0 kWh·units
EV charger	7.2 kW	14.4 kWh·units

Effect on RL: The RL agent does not directly see the power_kw value. It makes decisions based on the forecast value alone. Power draw only affects the savings meter displayed to the user after the session.

Input 3 — Duration Needed (hours)

```
"Hours of runtime needed (e.g. 1, 2, 3): "
```

What it asks: How many total hours does this task need to actually run — not how many hours it will be scheduled across the day.

What it is used for: This is the core of the task-based scheduling model. The RL agent will run the task for exactly this many hours total. Once the task has accumulated this many run-hours, the agent stops scheduling it for the rest of the day. The decision logic checks session._runs_so_far >= task.duration_hours and forces action = 0 for all remaining hours.

Effect on deadline enforcement: If there is not enough time left before the deadline, the agent is forced to run the task every remaining hour. Example: if duration=3 and the task has only run once by hour 19 with deadline=22, the agent must run at hours 20 and 21.

Realistic values by task type:

- Washing machine: 1–2 hours (a cycle typically takes 1–1.5 hours)
- Dishwasher: 1–2 hours
- EV charger: 2–8 hours (depends on battery level and charger speed)
- Dryer: 1 hour
- Pre-cooling HVAC: 1–3 hours

Input 4 — Deadline (hour)

```
"Deadline — finish by hour (0-23, e.g. 7 = 7 AM): "
```

What it asks: By what hour of the day must the task be completely finished. All duration_hours of runtime must be accumulated before this hour.

What it is used for: Deadline enforcement in the scheduling session. The agent checks two conditions each hour:

1. If (deadline - current_hour) <= runs_still_needed, it forces the task to run — there is no slack left.
2. If current_hour >= deadline and the task is still incomplete, it forces the task to run until complete.

Examples:

- User wants laundry done before leaving at 8 AM → deadline = 8
- User wants EV charged before morning commute at 7 AM → deadline = 7, duration = 4
- User wants dishwasher done before going to bed at 23:00 → deadline = 23

Input 5 — Interruptible

```
"Can this task be paused and resumed? [Y/n]: "
```

What it asks: Can the system split the runtime across non-consecutive hours, or must it run in one unbroken block?

What it is used for: Controls the contiguous-block logic in SchedulingSession. If interruptible = False, once the agent starts running the task it will not stop until all duration_hours are

complete. If interruptible = True (default), the agent can run hour 2, skip hour 3, and run hour 4 — the hours need not be adjacent.

Effect on savings: An interruptible task gives the agent more flexibility to find the cheapest hours across the day. A non-interruptible task may be forced into a less optimal window if the cheapest hours are not consecutive.

Realistic guidance:

- Washing machine: NOT interruptible — pausing mid-cycle damages fabrics and uses more water
- EV charging: Interruptible — most EVs handle paused charging gracefully
- Dishwasher: NOT interruptible — food residue dries if the cycle is paused
- Pre-cooling HVAC: Interruptible — can be paused and resumed without issue

Input 6 — Comfort Priority

```
[1] High Savings  — aggressive deferral  (penalty = 2.0)
[2] Balanced        — moderate tradeoff (penalty = 1.0)
[3] High Comfort  — rarely defer        (penalty = 0.2)
```

What it asks: How much the user values convenience versus cost savings. This is expressed as a comfort penalty weight.

What it is used for: The comfort_penalty is subtracted from the RL agent's reward whenever the agent skips a 'must-run' hour (hours 7, 8, 18, 19, 20). A higher penalty makes skipping those hours very costly in the reward function, so the agent trained with high penalties will have learned to avoid skipping them.

Important note: The RL agent is pre-trained. The comfort_penalty in the interface does not re-train the agent. It is used in the savings calculation to penalise must-run hour skips post-hoc. The agent's actual Q-table was trained with a fixed penalty during training (comfort_penalty=2.0 in LoadSchedulingEnv by default).

Input 7 — NLP Query Mode (Optional)

```
"Open NLP query mode? [y/N]: "
```

What it asks: Does the user want to ask free-text questions about the demand forecast before scheduling begins?

What it is used for: This opens an interactive mode where the user can type any text and the PeriodClassifier will predict whether it describes a peak, off-peak, or low demand period. The classifier also returns confidence probabilities for each class.

Example interactions:

- Query: 'Is 6pm a peak period?' → Classification: PEAK, Confidence: ~85%
- Query: 'What about midnight?' → Classification: LOW, Confidence: ~90%

- Query: 'Morning hours 10am to 12pm' → Classification: OFF-PEAK, Confidence: ~78%

This is the only place in the system where the user directly interacts with the PeriodClassifier. Everywhere else in the scheduling loop, the classifier receives generated text from the system itself.

Input 8 — Per-Hour Override

```
"Hour 18:00 | Override? [ENTER/0/1/optout]: "
```

What it asks: At each of the 24 hours, the user can accept the RL decision (press Enter) or force a different action.

Options:

- ENTER — accept the RL agent's decision as-is
- 1 — force the task to run this hour, regardless of what the RL agent recommends
- 0 — force the task to defer this hour, regardless of what the RL agent recommends
- optout — disable all automation for the rest of the day; the task will run every remaining hour

Effect on savings: Manual overrides are recorded with source = 'MANUAL OVERRIDE by user'. If the user forces a run during a high-demand hour, the savings for that hour will be zero or negative. The action log records both the rl_action (what the agent wanted) and the final_action (what actually happened), allowing the user to compare.

7. How Results Affect the Outputs

7.1 The Five System Outputs

Every hour, the system produces five pieces of output. Here is exactly what each one contains and how it is determined:

Output A — Period Classification Label (PeriodClassifier)

Stored in result['period_label']. This is the three-class category (peak, off-peak, or low) that the PeriodClassifier assigns to the current hour. It is determined entirely by the text

generated from the hour and forecast kWh value. It appears in the terminal display and in the action log.

Effect on other outputs: None directly. The period label is informational only — it does not change the RL decision. However, it contextualises the decision for the user: seeing 'PEAK' next to a 'DEFER' decision makes the reasoning obvious.

Output B — NLP Summary Sentence (ForecastSummarizer)

Stored in result['summary']. The plain-English explanation of the decision. Uses the real kWh forecast and the RL action. Appears after each hourly decision in the terminal and is saved to the action log.

Effect on other outputs: None. It is a read-only explanation.

Output C — RL Scheduling Decision

Stored in result['rl_action'] (raw RL recommendation) and result['final_action'] (after override and deadline enforcement). This is the binary decision: 1 = run, 0 = defer.

What determines it:

- The RL agent's Q-table, trained on the scaled forecast value, hour_sin, hour_cos, and the previous action
- Must-run hour enforcement: hours 7, 8, 18, 19, 20 are always forced to 1
- Deadline enforcement: runs the task if there is no slack left before the deadline
- Task-complete logic: forces 0 once duration_hours is accumulated
- User override: replaces the RL decision if the user typed 0 or 1

Output D — 24-Hour Forecast Table

Shown once before scheduling begins. Lists each of the 24 hours with its forecast kWh, period category, and load level (HIGH/MED/LOW). The deadline hour is marked. This gives the user a preview of the full day before any decisions are made.

Output E — Savings vs. Baseline Meter

Shown at the end of the session. Calculates total savings compared to an always-on baseline (task running every hour for 24 hours). The formula per hour is:

```
cost_always_on = forecast_kWh × power_kW
cost_agent    = forecast_kWh × power_kW × final_action
savings_step   = cost_always_on - cost_agent

# If this was a must-run hour and the agent skipped it:
```

```
savings_step  -= comfort_penalty
```

The savings bar shows the cumulative total as a percentage of the always-on baseline cost. A task that never runs would show 100% savings — but it would also be incomplete, which is penalised separately.